Oracle EBS Support Agent

Demo Flow

Analyzing and Resolving L1 and L2 Tickets with JIRA

A twelve-minute walk-through that demonstrates how the agent handles support tickets in both directions. Inbound tickets get diagnosed and answered with structured evidence from live EBS. Outbound, the agent detects problems before they are reported and creates fully researched tickets with recommended fixes. Both directions use the same tool surface, the same audit trail, and the same JIRA integration.

Duration	12 minutes
Audience	Support leadership, ops managers, EBS owners
Prerequisites	Flask app running on port 8000. JIRA credentials configured. Demo mode is fine.
Scenes	Four scenes plus opening and closing
Outcome shown	Two real tickets in JIRA, one diagnosed and one created from scratch

WHY THIS FLOW

The strongest demo narrative is the contrast between the two modes. Most support teams only have a reactive workflow: a ticket arrives, a human investigates, a fix is proposed. Showing both reactive and proactive in the same twelve minutes lands the message that the agent is not a chatbot bolted onto JIRA. It is the same engine driving both directions, with the audit trail to back every action.

Setup before you start

Five minutes of preparation makes the difference between a smooth demo and a stalled one.

- Flask app running on port 8000. Visit http://localhost:8000/jira in advance to warm up the singletons.
- Three browser tabs ready: the JIRA page in this app, the JIRA web UI at u2xai.atlassian.net, and the chat page.
- Pre-create one JIRA ticket with a realistic title such as *'Period close failing. 45 invoices stuck unposted'*. Assign it to support.
- Do not open SUP-2 in front of the audience. Its first comment was generated before the formatting fix and contains em-dashes.
- If demoing on a laptop, plug in. Every Claude call is 2 to 5 seconds and battery throttling makes that worse.

SCENE 1 — Reactive flow. An L1 ticket comes in. — 4 minutes

NARRATIVE

A support engineer starts their day. There is a P2 ticket waiting in JIRA. The customer is asking why the period close is blocked.

WALK-THROUGH

1. Open **/jira**. Search for the pre-created ticket. Click the row.
2. Drawer slides open with the description and any prior comments.
3. Click **Troubleshoot in EBS**.
4. While Claude is working, narrate what is happening on screen: the model is classifying the ticket against the catalog of 15 Agentic Apps, picking one to three, then running them against live Oracle EBS.
5. Findings appear with severity chips, counts, and the rule that fired. The AI report renders below with Summary, Evidence, Likely root cause, Recommended next steps, and Confidence.
6. Click **Post Report as JIRA Comment**.
7. Switch to the JIRA web tab and refresh. Show the comment with proper headings, bold EBS table names, and a numbered list of fixes. No literal markdown characters.

WHAT THIS PROVES

An L1 engineer who used to spend 1 to 2 hours diagnosing gets a structured root cause and a recommended fix in under thirty seconds, posted back to the same ticket the customer is watching. The engineer stays in the loop. The agent does the legwork.

SCENE 2 — Proactive flow. L2 finds issues before they are reported. — 4 minutes

NARRATIVE

The best L2 work is catching problems before anyone files a ticket. The agent can run that pass on demand.

WALK-THROUGH

1. Back to **/jira**. Click **Scan EBS for Errors** in the hero.
2. Modal opens. Confirm the look-back window. Click **Run Scan**.
3. Show the results table sorted by severity. Twenty-six findings across the 15 Agentic Apps.
4. Pick one CRITICAL row, for example Payables unposted invoices.
5. Click **Generate & Create**.
6. Claude drafts the ticket: summary, sectioned description, suggested priority Highest, labels, issue type Bug. Walk the audience through what the AI included: the actual SQL evidence, the root cause analysis citing real EBS tables, the six step fix plan, and real Oracle MOS doc references.
7. Briefly edit one field to show that the human stays in control. Add a label or tweak the priority.
8. Click **Create JIRA Ticket**.
9. Toast appears with the new ticket key. Click **Open**. JIRA web tab opens the new ticket. Show the formatting: real headings, bold table names, numbered fix list, MOS docs as a bullet list.

WHAT THIS PROVES

The same engine flips direction. No human had to find the problem. The ticket arrives in JIRA fully researched, prioritised, and ready for an engineer to action. The research that an L2 engineer would have done over a half day is in the description from the start.

SCENE 3 — Composability. Pull the same evidence into chat. — 2 minutes

NARRATIVE

When an engineer wants to dig deeper, the same data is available conversationally. The JIRA page, the Scan modal, and the chat all share one tool surface.

WALK-THROUGH

1. Switch to the chat tab.
2. Ask: *'Show me the top five invoices on hold for the past 30 days, with vendor names.'*
3. The agent uses the MCP tool payables_list_holds. A data table renders inline in the chat.
4. Follow up: *'For invoice 80123, give me the supplier 360 view.'*
5. Tool chain runs. Supplier profile, recent bank changes, and aging buckets all return.

WHAT THIS PROVES

The integration is composable. The chat surface, the JIRA page, and the fusion apps all call the same 61 MCP tools. Whatever the engineer prefers, they get the same data and the same answers.

SCENE 4	Audit trail. Every step is logged.	1 minute

NARRATIVE

AI assistance only works in production if the work is traceable. Every tool call, every SQL query, every JIRA write is observable.

WALK-THROUGH

1. Open **/observability**.
2. Show the agent flow strip and the recent run events stream.
3. Highlight the per-agent state grid and the in-flight pill.
4. Note: every AI step, every SQL query, every JIRA write is logged. Compliance and reproducibility are not an afterthought.

Closing

Wrap with three numbers and a single sentence per number.

- **One system** handles both reactive (ticket in) and proactive (ticket out). The same Claude orchestrator, the same MCP tools, the same JIRA writer.
- **15 Agentic Apps and 61 MCP tools** behind the AI. All callable from chat, from the JIRA page, or from Claude Desktop over stdio.
- **Time per ticket**: a 1 to 2 hour manual diagnosis becomes 30 seconds of AI work plus engineer review. The savings compound across the team.

Things to watch out for

- Demo on a fast network. Each Claude call is 2 to 5 seconds. Dead air feels long on stage. Have something to say during the wait.
- If JIRA latency spikes the Troubleshoot button looks frozen. Have a backup screenshot of a completed report.
- Do not open SUP-2 live. Its first comment has em-dashes from before the formatting fix. Use a fresh ticket created during the demo.
- Keep the audience focused. Resist the urge to detour into the Supply Chain Planning page or the analyzer registry. They are real but they are not this story.
- Have an answer ready for cost questions. Each end-to-end resolution is roughly one Sonnet call for classify, one for synthesis, and one for draft. Cheap relative to an engineer hour.

Appendix. Capabilities exercised in this demo

For an audience that wants the technical surface, here is the exact set of capabilities the demo touches.

MCP TOOLS CALLED

- **jira_get_issue** and **jira_search_issues**. Read the ticket and similar prior issues.
- **jira_scan_ebs_errors**. Run all 15 Agentic Apps and rank ticketable findings.
- **jira_generate_ticket_draft**. Claude drafts a fully sectioned ticket from one finding.
- **jira_troubleshoot_issue**. Classify, run the chosen apps, synthesize a report.
- **jira_create_issue**, **jira_add_comment**, **jira_update_issue**. Writes to JIRA with markdown-to-ADF conversion.
- **payables_list_holds**, **payables_get_supplier_360**. Composability demo in chat.

PAGES TOUCHED

- **/jira**. The main demo surface. Scan, draft, create, troubleshoot.
- **/**. The chat surface for the composability scene.
- **/observability**. The audit trail.

OUTPUTS THE AUDIENCE SEES

- A JIRA comment posted to an existing ticket, with proper formatting.
- A new JIRA ticket created from a scan finding, fully researched.
- Inline data tables in the chat answering ad-hoc support questions.
- Real-time observability of every AI step.